Assignment #2

Simon Leonard

February 26th 2026

1 Introduction

In this assignment, you will implement and test a hand-eye calibration algorithm. You will manually move the robot to several configurations to observe the augmented reality (AR) tag positioned on the floor. Most of this assignment (~90%) can be done without using ROS or any robot. You only need ROS to collect data at the end. Although this document uses illustrations from ROS for instruction purposes.

The position/orientation of the AR tag with respect to the camera is automatically computed and the position/orientation of the end-effector (forward kinematics) is provided by ROS. Your task is to use these measurements to test your algorithm and compare your results to the hand-eye transformation used by the simulation.

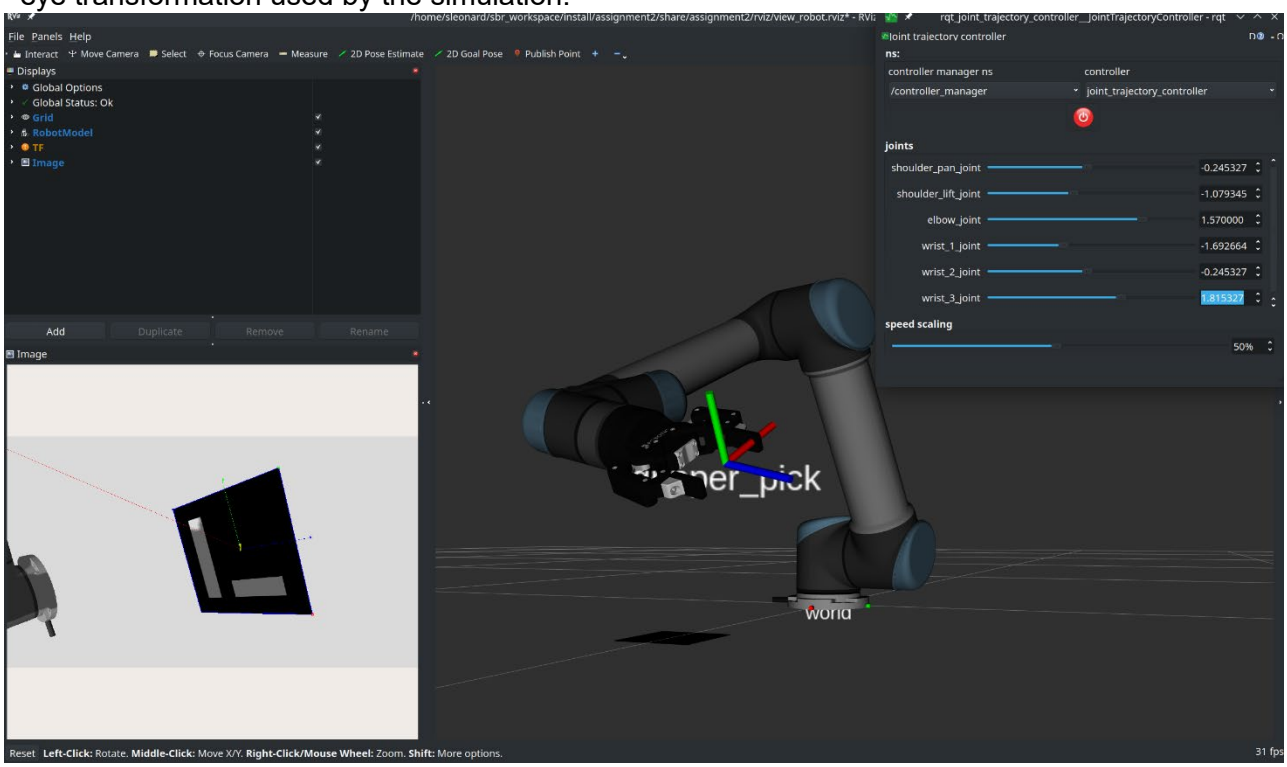


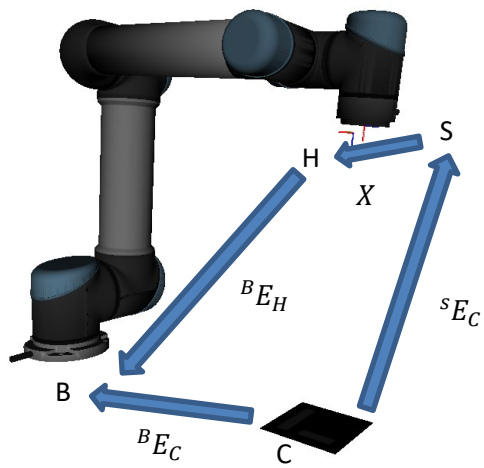
Figure 1 Rviz displays the camera image with the AR tag detected. The coordinate frames are also displayed in the right panel.

1.1 Deliverables

1. A Matlab file that generates synthetic data
2. Matlab files that implement the hand-eye calibration with all the required functions
3. File with your collected data

2 Warmup Exercise (1pt)

In this exercise, you will generate synthetic data to test your hand-eye calibration implementation in Matlab. First, let us name the frames that we will use as follow: “B”ase, “H”and, “S”ensor, “C”heckerboard. Note that we have the equation ${}^{B}E_C = {}^{B}E_H \, X \, {}^{S}E_C$



The purpose is to generate several random, noise-free A_i and B_i that you can use to validate your hand-eye calibration implementation. You will do this by assuming that you know the transformation ${}^{B}E_C$ (in practice you don’t know this so you will make one up).
Then you will generate a random transformation X and N random transformations ${}^{B}E_H$. Finally, for each ${}^{B}E_H$, you will compute the missing transformation ${}^{S}E_C$. These ${}^{B}E_H$ and ${}^{S}E_C$ can be combined to compute the Ai and Bi used to solve $AX = XB$ (with the advantage that they are noise-free and you don’t need to move any robot!). The data you generate in this section can (and should) be used to validate your hand-eye calibration implementation. Collecting real data (either a real or simulated robot) is long, subject to noise and prone to errors. The data you generate in this section is “perfect”, no noise and without errors. Use it to test your code. If your hand-eye calibration returns ***exactly*** the matrix *X* that was used to generate the A_i and B_i, your code is good. If not, then your code is wrong (and you cannot blame “bad data” for the errors).

In Matlab create a file `generatedata.m` and complete the following two functions. Pay attention to the output format outlined below as it matches the one used by ROS.

```
% Generate synthetic data to test hand-eye calibration.
% e_bh and e_sc are Nx7 matrices that represent N E_bh and N E_sc
% transformations. Each row is of the form [ tx ty tz qx qy qz qw ]
% were tx, ty and tz denote a translation and qx, qy, qz, qw a
% quaternion.
% X is a randomly generated hand-eye transformation
function [e_bh, e_sc, X] = generatedata(N)

   % Assume we know the transformation between the world and the
   % checkerboard
   E_bc = [ eye(3) [ 1; 0; 0 ]; 0 0 0 1 ];

   % Create a random X for generating the data
   X = randSE3();
   e_bh = [];
   e_sc = [];
```

```
  for i=1:N
    % Now that you have X and E_bc
    % TODO: Generate a random E_bh, you can use rotm2quat to convert
    %       the rotation matrix to a quaternion (be careful because it
    %       outputs in the format [qw, qx, qy, qz]) and append the
    %       transformation to e_bh

    % Now that you have X, E_bc and E_bh
    % TODO: Find E_sc and append the transformation to e_sc
  end

% Generate a random SE3 transformation
function Rt = randSE3()
    % TODO: Generate a random rotation matrix
    % R =
    % TODO: Generate a random translation
    % T =
    Rt = [ R t; 0 0 0 1];
```

3 Installation of the SBR package

Download and run the following script. It will create your ROS workspace for this course and compile the dependencies.

```
[sleonard@merliver ~]$ wget www.cs.jhu.edu/~sleonard/sbr-a2.sh
[sleonard@merliver ~]$ chmod 755 sbr-a2.sh
[sleonard@merliver ~]$ ./sbr-a2.sh
```

The script will download and build the ROS Industrial packages for the UR-5 and the aruco_ros used to track the AR tag.

3.1 GUI animation of the robot

You will use the same GUI from assignment #1 to simulate and move the robot. The main difference in this assignment is that a camera is mounted on the end-effector of the robot. To start the simulation, use the command

```
[sleonard@merliver ~]$ ros2 launch assignment2 assignment2.launch.py
```

Move the robot until the camera sees and detects the AR tag as illustrated in Figure 1. Rviz will display the camera images and, if the AR tag detection succeeds, the image will display the tag's name and its coordinate frame.

Your first task is to implement the exact (closed-form) solution of Park and Martin. For this, you will need to move the robot to three configurations to collect two As and two Bs while ensuring that the AR tag is properly detected. For each configuration, you will use `tf2_echo` to query and save to a text file the position/orientation of the `gripper_pick` with respect to the `world` (forward kinematics) and the position/orientation of the frame `marker` with respect to the frame `camera_link` (these correspond to the transformations ${}^{B}E_{H}$ and ${}^{S}E_{C}$ in Section 2). Note that `tf2_echo` reports rotations with a quaternion representation $[q_x\ q_y\ q_z\ q_w]$. If your Matlab version has the robotics toolbox you can use the functions `rotm2quat` and `quat2rotm` to convert rotation matrices to quaternions and vice versa (these functions use $[q_w\ q_x\ q_y\ q_z]$ to represent

quaternions). Be careful with the order: Matlab and ROS use a different order. ROS uses the format **[q_x q_y q_z q_w]** whereas Matlab uses **[q_w q_x q_y q_z]** and your code need to convert between them.

Your second task is to implement the ***least-squares version*** of Park and Martin hand-eye calibration algorithm. For this, you will need to move the robot to several configurations (N>3) while ensuring that the AR tag is properly detected.

You need to write Matlab functions to parse the transformation data you saved and implement the closed form and least-squares versions of Park's hand-eye calibration algorithm (AX=XB). You can use the following command to visualize your results and compare them against the values used by the simulator

```
$ ros2 run tf2_ros static_transform_publisher --x X --y Y --z Z --qx QX
        --qy QY --qz QZ --qw QW --frame-id gripper_pick --child-frame-id result
```

where you need to replace `X Y Z QX QY QZ` and `QW` with the values you obtained from your result. This will display your frame in Rviz and the result should be close to the real camra.

4 Implementation guidelines

You will need to implement the following Matlab functions. Our TAs will automate scripts to test your code and their scripts will expect your functions to have the *exact* signatures outlined below. **Each of these functions must be in its own file. A function that fails a test because because you decided to use different parameters or array sizes will be graded with "0"**. Save your files in the directory assignment2/matlab.

HINTS: Matlab has several functions to make your life easier. Please have a look at the functions `logm`, `power` and `mldivide` (aka "\").

4.1 Unique Solution (1pt)

Create a file axxb_closedform.m and implement the following function

```
function X=axxb_closedform( e_bh, e_sc )

e_bh: a 3x7 matrix that contain 3 forward kinematics measurements
      obtained from tf_echo. The format of each row must be
      [t_x t_y t_z q_x q_y q_z q_w]
e_sc: a 3x7 matrix that contain 3 AR tag measurements obtained from
      tf_echo. The format of each row must be [t_x t_y t_z q_x q_y q_z q_w]
return: the 4x4 homogeneous transformation of the hand-eye
        calibration
```

This function uses three measurements to create A1, A2, B1 and B2 and to calculate X. Use the close form solution in this function.

4.2 Least-Squares Solution

1. Function SolveRx (2pt)

Create a file solveRx.m and implement the following function

```
function Rx=solveRx( alphas, betas )
alphas: A 3xN matrix representing the skew symmetric matrices. That
        is, alphas = [α_1 ··· α_N]. Each α_i is a 3x1 vector
betas: A 3xN matrix representing the skew symmetric matrices. That
       is, betas = [β_1 ··· β_N]. Each β_i is a 3x1 vector
return: The least squares solution to the 3x3 matrix Rx.
```

Make sure that your solveRx works before moving on. You can use your synthetic data from Section 2 (this is "perfect data" without noise and should produce perfect results). By using this data, you should get exactly the rotation Rx that was used to generate the data. Don't bother moving on to 4.3 if you cannot validate Rx, because it will not work.

2. Function solveTx (2pt)

Create a file solveTx.m and implement the following function (pay attention to the dimension of the parameters)

```
Function tx=solveTx( RA, tA, RB, tB, RX )

RA: a 3x3xN matrix with all the rotations matrices R_Ai
tA: a 3xN matrix with all the translation vectors t_Ai
RB: a 3x3xN matrix with all the rotations matrices R_Bi
tB: a 3xN matrix with all the translation vectors t_Bi
RX: the 3x3 rotation matrix R_x
return: the 3x1 translation vector t_x
```

3. Function axxb (2pt)

Create a file axxb.m and implement the following function

```
function X=axxb( e_bh, e_sc )

e_bh: a Nx7 matrix that contain N forward kinematics measurements
      obtained from tf_echo. The format of each row must be
      [t_x t_y t_z q_x q_y q_z q_w]
e_sc: a Nx7 matrix that contain N AR tag measurements obtained from
```

```
      tf_echo. The format of each row must be [t_x t_y t_z q_x q_y q_z q_w]
return: the 4x4 homogeneous transformation of the hand-eye
        calibration
```

This function does everything. It preprocesses the input transformations and calls the other functions to solve for the hand eye calibration transformation X.

HINT: To get the 3x3xN rotation matrices, you can use the `quat2rotm` function with all the quaternions at once (i.e `quat2rotm( [ 0 0 0 1; 1 0 0 0 ] )` will return a 3x3x2 matrix).

5 Expert Challenge (1pt)

Collect data from the simulation and submit it with your code. You will get 1pt if you achieve less than 5mm for translation error (i.e. Euclidean norm) and less than 0.1 radian for rotation error. For the angle error, use the angle of the following rotation matrix $R_x \hat{R}_x^T$, where R_x is the true rotation (as reported by tf_echo) and $\hat{R}_x^T$ is the transpose of your solution. Use as many A_i and B_i as you want/need.

6 Submit your code and data

1. Make sure you saved all your Matlab files (`axxb.m`, `axxb_closedform.m`, `solveRx.m`, `solveTx.m` and `generatedata.m`) in the directory assignment2/matlab.
2. Also submit the data that you collected to test your implementation. The test data should be in a text file `assignment2/matlab/data.txt` with the following format (including the # lines).

```
# gripper
tx1 ty1 tz1 qx1 qy1 qz1 qw1
…
txN tyN tzN qxN qyN qzN qwN

# marker
tx1 ty1 tz1 qx1 qy1 qz1 qw1
…
txN tyN tzN qxN qyN qzN qwN
```

3. You must create and submit your assignment2.tgz package by using the following command (marks will be deducted if your package is not created as follow)

```
tar cvfz ~/assignment2.tgz -C ~/sbr_workspace/src/sbr assignment2
```

Reminder: By submitting your assignment you acknowledge that you have read the JHU Academic Misconduct Policy and that you are the author of 100% of the code submitted. Furthermore, distributing or sharing your assignments (whereas online, with classmates or students that will take this course in future semesters) is not authorized and will be severely sanctioned.